Studio

A product photo and a brief in, a finished 3D ad out.
Architecture and how it works.

A multi-agent pipeline built for the Meta Global AI Developer Hackathon (October 27 to November 4, 2026) on Meta's models: Muse Spark, Muse Spark Open and SAM 3.1. AI agents read the brand from one photo, plan the ad, drive a real-time PBR renderer, and check their own work frame by frame, revising until the ad passes.

Repository	github.com/Mar-Dev-One/Meta-hackathon
State described	commit d90a5c0 and the work after it (3D model products), October 29, 2026
Audience	developers and reviewers who want to understand or extend the system

Contents

1. Overview 3

2. Repository layout 4

3. The Studio renderer 5

- Command server . . . 5
- Framing . . . 5
- Rendering . . . 5

4. The brand analyst 6

5. The director 7

- Writing . . . 7
- Revising . . . 7
- Storyboard . . . 7

6. The critic 8

- Measured checks (exact) . . . 8
- Judgment (vision) . . . 9
- The loop . . . 10

7. Final cut 11

8. Products of any shape 11

- The model catalogue . . . 11
- Fitting and dressing . . . 11

9. The web app 12

10. Data and runs 12

- Model clients . . . 13

11. Results 14

- Muse Spark vs Muse Spark Open as the critic . . . 14

12. Known limits 15

13. Built during the hackathon 15

- Credits . . . 15

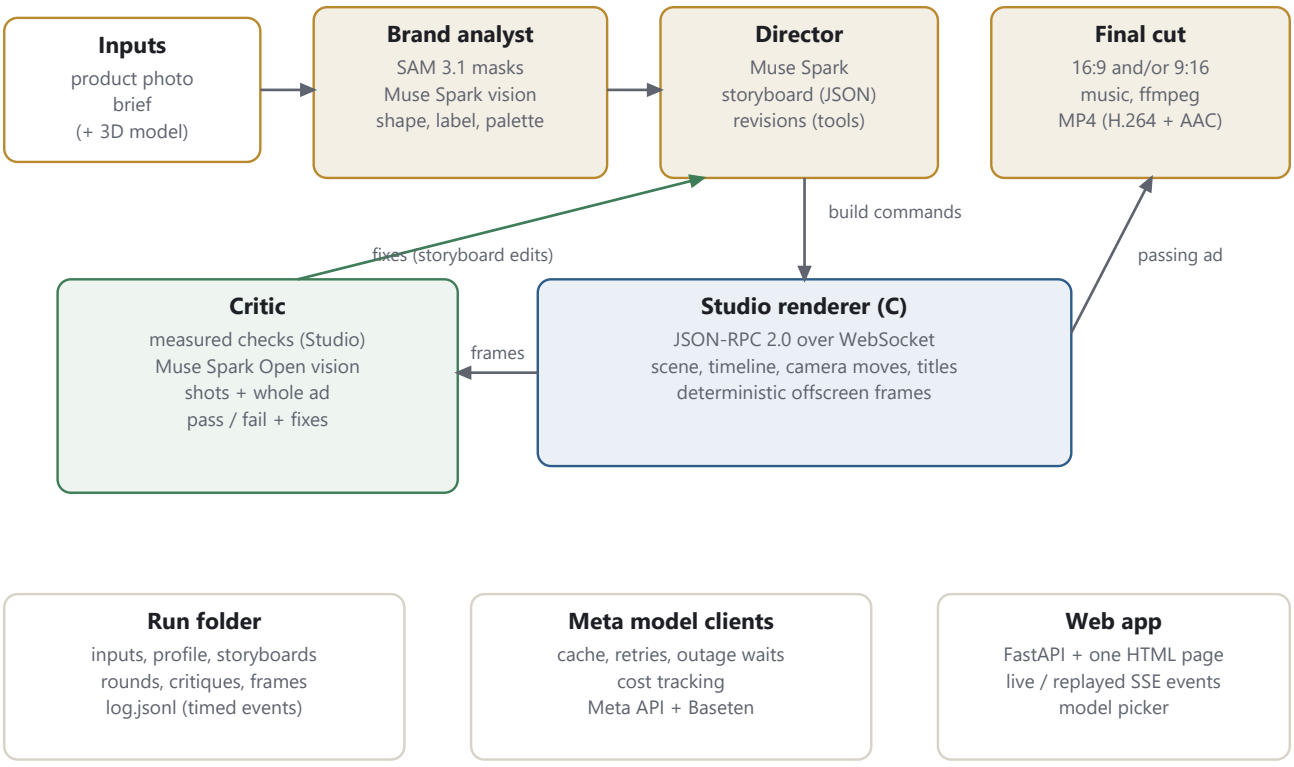
1. Overview

Studio turns **one product photo** and **a one-line brief** (for example *"Luxury perfume ad, dark and moody, 15 seconds"*) into a finished marketing video, horizontal (16:9), vertical (9:16) or both, with titles and music. Nothing is filmed: the product is rebuilt in 3D and every frame is rendered.

Five stages, each an agent or a program with a clear contract:

- **Brand analyst**: reads the photo. SAM 3.1 segments the product, its cap and its label; the outline becomes a 3D shape, the label is cut out and unrolled, the palette is measured, and Muse Spark reads what pixels can't (brand, materials, typography, mood, what an ad must avoid). Output: a *brand profile*.
- **Director**: plans the ad as data. Muse Spark writes a *storyboard* (look, shots, titles, animation) in a strict JSON schema; code checks the craft rules and sends problems back.
- **Studio**: a C renderer driven over a command server. It builds the set and the product, keys every camera move on a timeline, draws titles, and renders deterministic frames at any size.
- **Critic loop**: measures each shot exactly, judges it with Muse Spark Open vision, judges the whole ad, and sends failing issues back to the director as storyboard edits; at most three revisions, the best round kept.
- **Final cut**: renders the passing storyboard in the chosen formats, adds music in the style's mood, and encodes MP4.

A web app starts runs, streams their progress live, and replays any finished run with its timing. Every run is a folder holding every input, output, frame, critique and a timed log.



Every stage writes into one run folder; the app streams that folder's log.

Figure 1. The system. The analyst hands the director a brand profile; green arrows are the critic loop.

2. Repository layout

Path	Language	What it is
src/main.c	C	The Studio program: window or hidden, command server setup, scene loop
Engine/	C	Meta_Engine, the Studio library: studio commands, timeline, camera moves and framing, lighting presets, products and backdrops (glTF), titles (SDF fonts), shot rendering; tests
Deps/	C (prebuilt)	PBR_Engine headers and libraries: the pre-existing renderer the Studio is built on
pipeline/meta/	Python	Clients for the Meta models and Baseten: cache, retries, outage waits, cost
pipeline/studio/	Python	Studio client; storyboard → commands; rendering; reframing; 3D model files
pipeline/schemas/	Python	pydantic models: BrandProfile, Storyboard, Critique, AdCritique
pipeline/agents/analyst/	Python	The brand analyst
pipeline/agents/director/	Python	Shot grammar, style library, writing and revising storyboards
pipeline/agents/critic/	Python	Measurements, cuts, shot and whole-ad judgment, the loop, model comparison
pipeline/agents/producer.py	Python	The whole chain: photo + brief → ad
pipeline/catalog/	Python	3D model search and download: Sketchfab, Objaverse, Poly Haven
pipeline/encode/	Python	ffmpeg finishing; the synthesized music
pipeline/demos/	Python	Five hand-made ads for fictional brands (test footage, examples)
pipeline/runs.py	Python	Run folders: log, outputs, replay, reopen
app/	Python + HTML	FastAPI backend and one HTML page
Assets/	data	Fonts (OFL), HDRIs (CC0), shaders, models, test product photos

Roughly 13,500 lines of project code: about 6,700 of C, 4,300 of Python and 2,400 of GLSL at the start of the critic phase, plus the app and later work. The rest of the tracked lines are the engine's vendored headers.

3. The Studio renderer

The Studio is a C program built on **PBR_Engine**, a physically based real-time renderer that predates the hackathon (PBR shading, image-based lighting, shadows, bloom, TAA, SSAO, an entity/component world and a command system). During the event, glass (transmission, IOR, thickness, absorption), depth of field, clear coat and specular were added to it. Everything that makes it a studio is new: the **Meta_Engine** library in `Engine/` and the program in `src/main.c`.

Command server

The Studio runs a WebSocket server on 127.0.0.1 speaking **JSON-RPC 2.0**. On start it writes `server.json` next to the exe with the URL and a per-run token. The method is the command, the params its arguments; a failed command changes nothing and says which argument is wrong and what it accepts. Every change is also pushed to clients as a `scene.changed` notification. `commands.list` describes every command, and the director's shot grammar is generated from it.

```
-> {"jsonrpc": "2.0", "id": 3, "method": "camera.move", "params": {"move": "orbit",
    "subject": "product", "start": 0, "duration": 4, "azimuth": -40, "to_azimuth": 40, "fill": 0.6}}
```

Commands	What they do
studio.reset, environment.*, render.*	Empty the stage; HDRI, exposure, bloom, tone mapping, SSAO; antialiasing, shadows
product.create	A product spun from a profile: a template (perfume, bottle, jar, can, dropper) or traced parts, with a label image wrapped round the body; parts can be glass, clear-coated or metallic
place_model	Any model file (.glb), sized in meters and placed; used for products of any shape
backdrop.create	A cyclorama (floor curving into a wall) or a round pedestal
lighting.preset	three_point, rim_dark, high_key, top_soft, split, neon_dual; glow lights glass from behind
camera.move	static, orbit, dolly, crane, truck, reveal, zoom: a shot's move as timeline keys, with depth of field
timeline.*	Keyframe any number, vector or colour field with easing; seek, play, pause
text.*	Titles drawn over every frame in five open fonts; measured; animatable
shot.preview, shot.render, shot.bounds	PNG frames; a time range straight to MP4; where an entity sits in the frame

Framing

`camera.move` places the camera so the subject fills **fill** of the frame's height as seen from the camera's elevation. It projects the product's top and bottom rims through a camera aimed at its centre and searches for the distance where the outline spans that share. Each edge also stays inside the frame up to fill 0.97; above that the product is cropped on purpose (a close-up). In a narrow 9:16 frame a wide product is held by the width: up to fill 0.7 its width may take 80% of the frame; past that the cap rises smoothly so a push-in reaches the size it asks for by cropping the sides. A truck's default slide is most of the free room beside the product, so it stays whole in any frame.

Rendering

Shots render offscreen at any size, independent of the window. Each frame is rendered from scratch at its time (timeline evaluated, TAA restarted, then N sub-frames), so a frame looks the same alone or in a sequence and the critic can re-check any single frame. On an Intel Iris Xe a 1080p frame takes about 0.4 s into a video. Renders are byte-identical within a session; a fresh session can differ by one level in a few pixels.

4. The brand analyst

Input: a photo and what the product is ("perfume bottle"). Output: a **BrandProfile**, the facts every shot must respect, and the 3D product.

- **Segmentation** (SAM 3.1): the product, then its cap and label, kept only where they lie on the product.
- **Shape**: the mask's half-width per row about the vertical axis, smoothed and simplified, scaled to the real height and split into body and cap: a lathe profile the Studio spins into a mesh.
- **Label**: cut from the photo and unrolled from the body's curve (the inverse of the Studio's wrap), with the photo's shading divided out.
- **Palette**: k-means in Lab space, part by part, one colour per family so light and shade don't count as colours; roles product, accent, label, text.
- **Reading** (Muse Spark vision, JSON schema): brand name, real height, each part's material (colour, metallic, roughness; glass as transmission holding a liquid's colour; clear coat), typography, mood and the brand's *do-nots*.



Figure 2. The analyst's contact sheet for a test product (fictional brand): masks, traced profile, turntable of the rebuilt product, unrolled label, palette and Muse Spark's reading.

Products that aren't round (phones, boxes, sculpted bottles) take a different path: a 3D model file found online or uploaded replaces the traced shape (section 8).

5. The director

Writing

Muse Spark writes a *plan* (the storyboard without the product, which comes from the brand profile) from the brief, the profile, a **style library** of eight complete looks (luxury_dark, clean_minimal, bold_energetic, natural_organic, warm_glow, tech_sleek, playful_pop, soft_pastel) and the **shot grammar**. The grammar is generated from the Studio's own command descriptions and the Storyboard schema, and checked against both, so the director is only told about things the renderer can do.

The answer is validated by the schema, then checked in code against craft rules a schema can't hold:

- shot lengths add up to the ad's length; each shot 1.2 to 5 s; consecutive shots move differently;
- the last shot is a still, front-facing hero with the brand's name appearing in it;
- the pedestal is in proportion with the product;
- titles fit the frame as the Studio measures them, don't overlap each other, and stay clear of the product in every shot they're on.

Problems go back to Spark with its previous answer, up to three tries. The chosen format (16:9 or 9:16) is given to the director, or read from the brief.

Revising

To answer a written fix (the critic's), Muse Spark calls tools that edit the storyboard, never the Studio: `edit_shot, add_shot, remove_shot, edit_look, edit_title, add_title, remove_title, edit_track, add_track, remove_track`. Each edit is checked against the schema as it's made; the craft rules when it's done. The shared light moves in small steps: exposure, ambient and light intensity at most ×1.35 each per revision and at most two of them at once, because every shot is lit by it. Every revision can be rebuilt, compared and undone.

Storyboard

```
{"title": ..., "brief": ..., "style": "luxury_dark", "aspect": "16:9", "fps": 30,
 "look":    {environment, background, backdrop, pedestal, lighting},
 "product": {template | profile_parts | model, parts / finish, label},
 "shots":   [{id, intent, duration, camera: {move, fill, azimuth, elevation, aperture, ...}}],
 "titles":  [{text, font, size, y, color, tracking, start, fade}],
 "tracks":  [{entity, component, field, keys: [{t, value, ease}]}]}
```

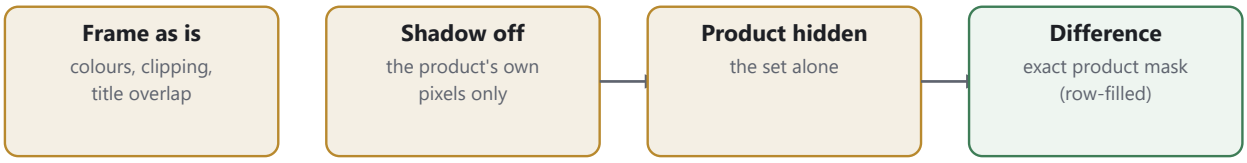
Field names follow the Studio's command arguments, so building a storyboard is mechanical (`pipeline/studio/build.py`).

6. The critic

The critic is what makes the output trustworthy. It **measures first and judges second**, and its verdicts are decided in code, not by the model.

Measured checks (exact)

For three moments in each shot (10%, 50% and 90% of it), the Studio renders the frame three ways:



Passes 2 and 3 are rendered with SSAO and bloom off, so the product changes only its own pixels.

Figure 3. Getting the product's exact pixels without a segmentation model.

Measure	What it says
fill, cut	The product's share of the frame's height and area; which edges cut it
centre, headroom	Where it sits; the space above it
edge lost	Share of its outline whose Lab distance across the edge is under 10: it merges with the background (a missing rim light)
separation	Lightness of the product against what's just around it
clipped / crushed	Shares of its pixels blown to white / crushed to black
palette	Distance from each storyboard colour to the nearest rendered colour (lightness at a third)
title overlap	Share of each visible title's box over the product

At every **cut** (last frame of a shot against the first of the next) it reads, from the Studio, how far the camera turns around the product (its own spin counted) and whether the set's lights change intensity or colour. The light is read from the lights, not the pixels: a new framing changes what's bright in the frame as much as a light does.

Intended effects are exempt: a reveal's product starting off frame, the lights fading up from dark, a close-up that crops the product (by height, or by width in a narrow frame).

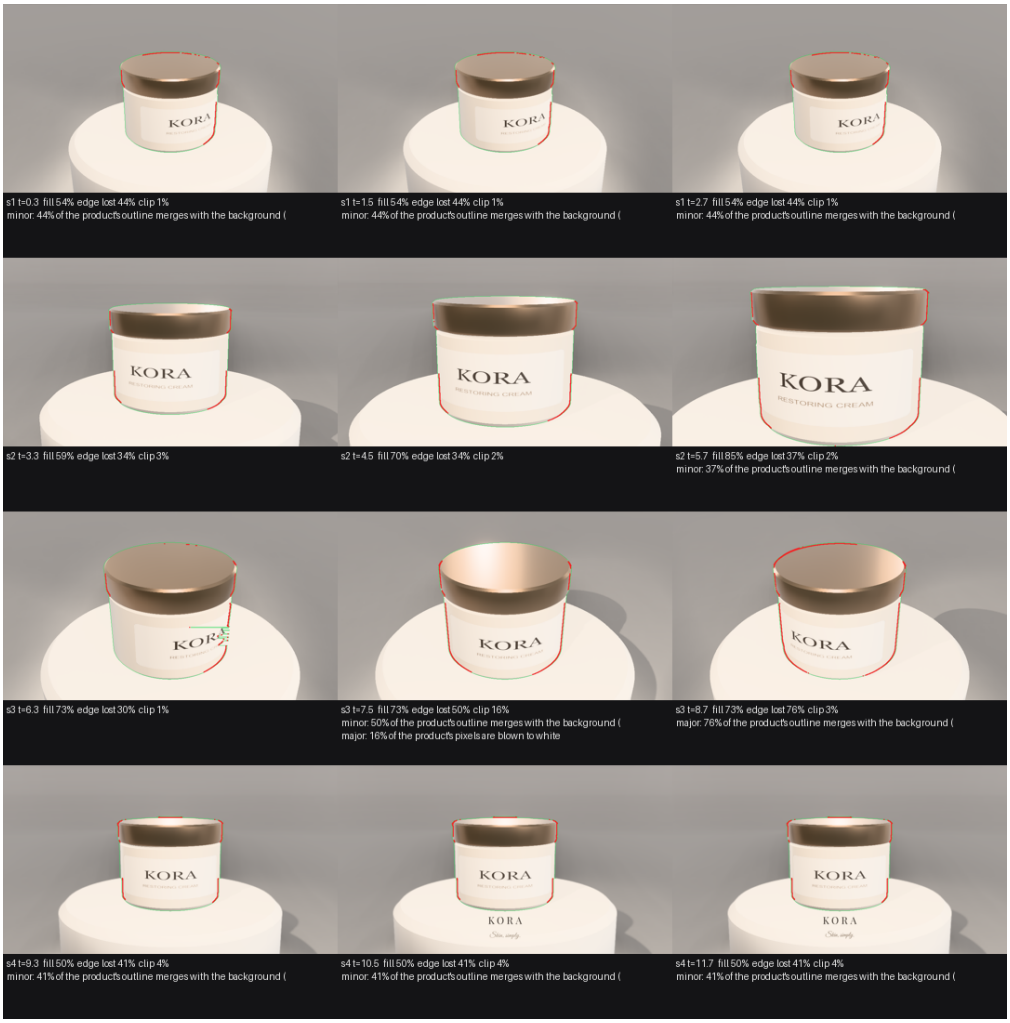


Figure 4. A contact sheet of measured frames: the product's outline in green, outline points that merge with the background in red (a cream jar on a cream set), the flags under each frame.

Judgment (vision)

Muse Spark Open (on Baseten) sees each shot's three frames with the measurements, the brief, the shot's intent and storyboard, the brand and the grammar. It scores composition, lighting, product visibility, brand, continuity and titles from 0 to 10 against written anchors, and writes issues with fixes in storyboard terms (`shots[1].camera.fill 0.9 -> 0.65`, `remove tracks[1]`).

- **Bound by the measurements**: a major measured flag must appear as an issue with that category scored 6 or less, or the answer goes back to the model.
- **Pass**: every score 7 or more and no major issue, decided in code.
- **Second opinion**: when the lowest score is 6 or 7, the shot is judged again as a new sample and the worse result kept (the same shot judged twice flips about a fifth of the time).
- **Whole-ad pass**, every round: a filmstrip and the cut pairs with the cut measurements; pacing, continuity, arc and brand. It catches what no single shot shows: a cut that swings to the product's back, a light that changes colour.

The loop

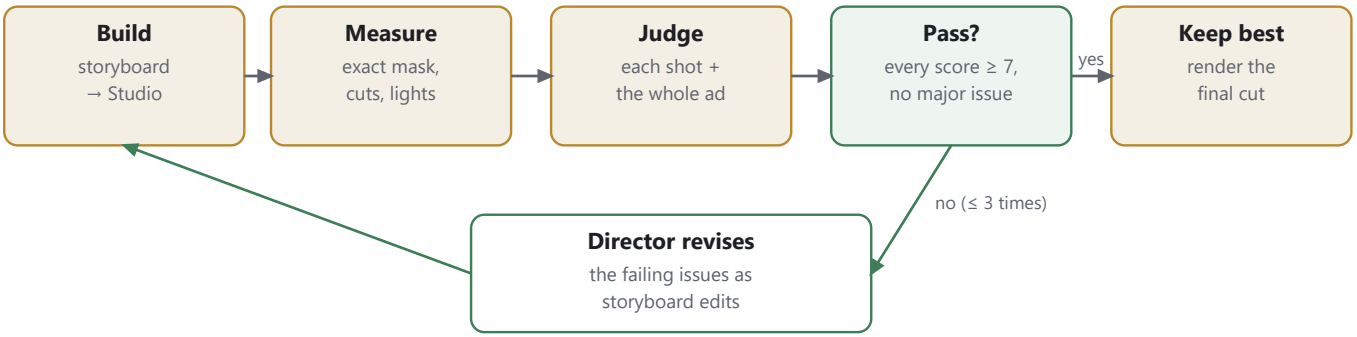


Figure 5. The critic loop.

Each round rebuilds the whole ad (the set is shared, so a lighting fix for one shot changes all of them), measures and judges every shot and the whole ad. If anything fails, the director gets the failing issues (the ad's and the shots'), the shots that passed (to leave alone), and what earlier rounds changed and how the scores moved, so it doesn't undo a fix. At most three revisions; if none passes, the best round is kept (most shots passing, the whole ad counting as one more, then the highest lowest score). A run that stops (a network error) resumes from its last finished round.

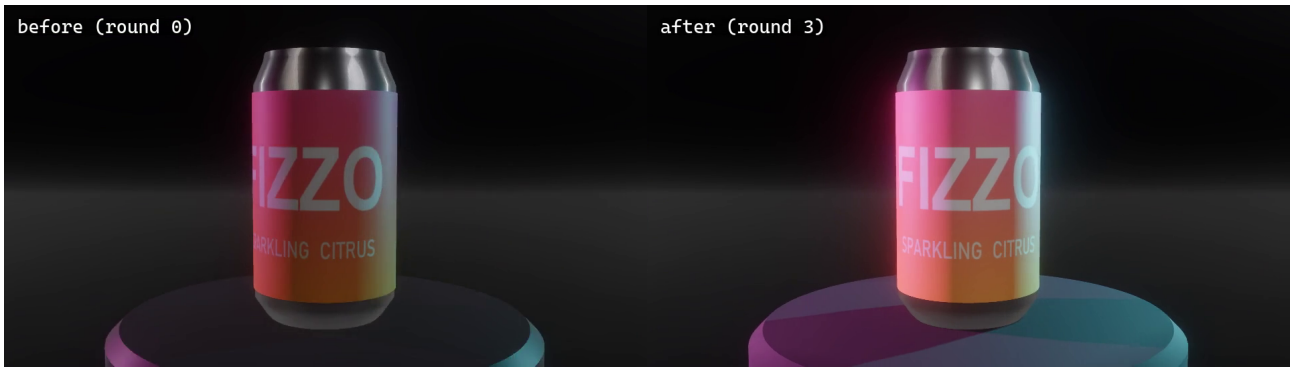


Figure 6. An ad broken on purpose (light down to a fifth, a slide that leaves the frame), round 0 against round 3: 1 of 5 shots passing to 5 of 5, with no human input.



Figure 7. A broken cut (left: the second cut swings to the back of the can under green and orange light). The whole-ad pass scored continuity 3; one revision fixed it (right: continuity 8, every shot and the ad passing).

7. Final cut

- **Formats**: horizontal 16:9, vertical 9:16, or both. With one format the director plans and the critic judges in it; with both, the brief's frame is planned and the other is **reframed**: titles are measured in the new frame and shrunk to fit 88% of its width, an explicit slide is scaled to the narrower frame, and the framing rules above keep the product right.
- **Rendering**: 1080p, 16 sub-frames per frame, straight into H.264 through ffmpeg; faded in and out.
- **Music**: synthesized note by note with numpy, so it's original and needs no licence. Four moods picked from the style: dark (minor pads, a sub drone, a heartbeat pulse), calm (soft pads and a plucked arpeggio), energetic (four on the floor, hats, bass, stabs), warm (electric-piano chords, a shaker). A swell lands on the hero shot's first beat. A licensed track at Assets/music/<mood>.mp3 replaces it.
- **Delivery**: MP4, H.264 CRF 16, +faststart, AAC 192 kb/s.

8. Products of any shape

The analyst's traced shape spins an outline around a vertical axis: right for bottles, jars, cans and droppers, wrong for a phone (it becomes a cylinder). For any other shape the product is a **3D model file**, found online or uploaded.

The model catalogue

Source	What it offers	Access
Sketchfab	The largest library; the most relevant results	Search open; download needs an API token
Objaverse (Allen Institute)	46,000 Sketchfab models in 1,156 categories; noisy	Downloaded from its Hugging Face dataset; licences checked on Sketchfab
Poly Haven	500+ CC0 models, high quality, few retail products; real-world sizes	Free API

One search queries all three in parallel and keeps only **CC0** and **CC-BY** models (changing a model's colours makes a derivative, and ads may be commercial), with at most 300,000 faces. Downloads and model details are cached; each download keeps its credit (title, author, link, licence), which the run saves as model_credit.json.

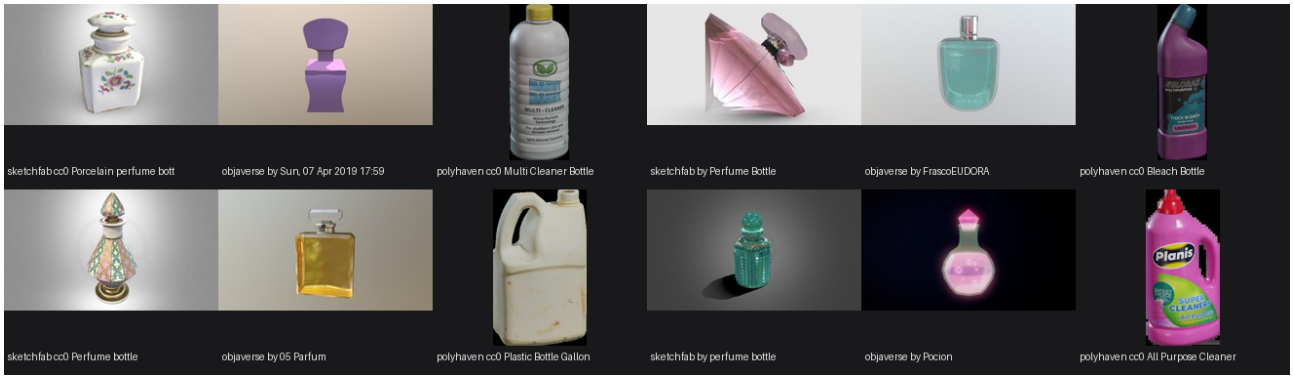


Figure 8. Search results for "perfume bottle" across the three sources, each with its licence. A person picks.

Fitting and dressing

- **The right way up**: of the six ways a model can stand, the one whose front-view proportions best match the product's outline in the photo (glTF's usual Y up wins unless clearly worse).
- **Real size**: centred, standing on its base, scaled to the analyst's height, with the transforms baked into the vertices (a glass's thickness is measured in its mesh's own units).

- **The brand's finish, part by part**: pieces centred in the top third get the cap's finish; the rest the body's. A texture is greyed and tinted with the brand colour, keeping its detail. A glass product becomes one glass tinted with its liquid's colour, like the Studio's own bottles: the renderer shows the scene behind a see-through surface but not other see-through surfaces, so liquid modelled inside a glass shell is removed.

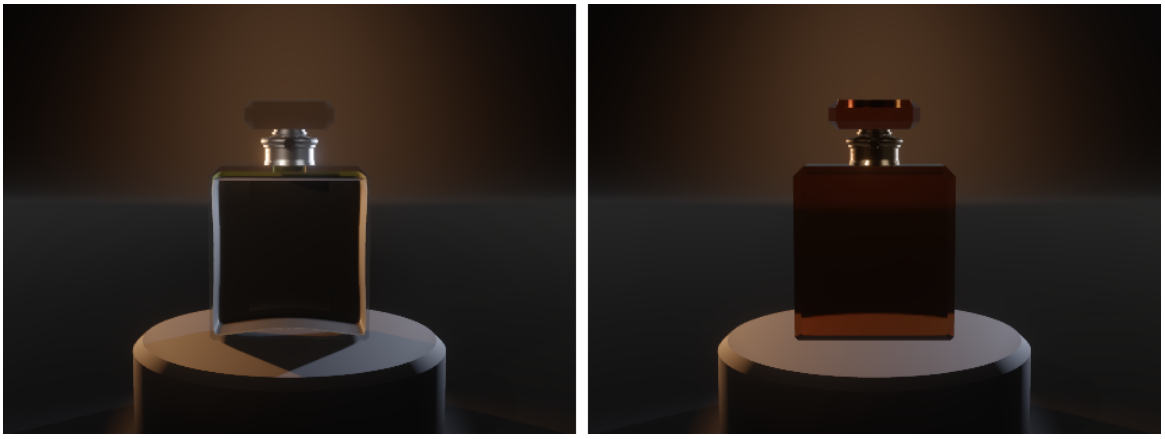


Figure 9. A downloaded perfume bottle as it came (left) and dressed in the brand's materials (right): amber glass and a gold collar.

9. The web app

`python -m app` serves a FastAPI backend and one HTML page (no build step) on http://127.0.0.1:8000.

Endpoint	What it does
GET /	The page
POST /runs	A photo, a brief, a category, the format, critic and video switches, and optionally a 3D model (a search result or a .glb upload); queues an ad and returns its id
GET /runs, GET /runs/{id}	Every run with status, brief, outputs, time and cost; one run's summary
GET /runs/{id}/events	The run's log as Server-Sent Events: followed live, or replayed with its timing (?speed=1, 2, 4)
GET /runs/{id}/files/{path}	Frames, contact sheets, videos; paths kept inside the run
GET /models/search?q=	3D models of a product from the catalogue

Runs go one at a time (one renderer, one GPU) through a worker thread with its own event loop and its own Studio: the agents' model calls block, and on the server's loop they would stall every stream. The worker writes the run's log; the streams read it, so a live run, a finished run and a replay are the same thing to a client.

The page fills in as events arrive: the brand profile card (photo, masks, palette, materials, mood, do-nots, the 3D model's credit), the storyboard as shot cards with a timing bar, the critic as a table of rounds by shots (click a cell for its issues and fixes, with each round's contact sheets), the videos with download links, and a live log with the running model cost.

10. Data and runs

Schema	Written by	Holds
BrandProfile	analyst	brand, category, height, shape (template, traced parts or model), materials, palette, typography, mood, do-nots, label
Storyboard	director	look, product, shots, titles, tracks, aspect, fps
Critique	critic	a shot's six scores, pass, issues with fixes, the measured numbers
AdCritique	critic	the whole ad's pacing, continuity, arc and brand; issues; the cut measurements

All are pydantic models with field descriptions written for the model that fills them; their JSON Schemas are exported for prompts. Unknown fields are errors, so a typo is never silently ignored.

```
runs/20261029-183824-ad-mira_jar/
  log.jsonl            every event with its time: model calls and costs, Studio commands, critiques
  run.json             status, length, model calls, cost, outputs
  upload/, inputs/     the photo (and model) as given
  brand_profile.json   masks.png, outline.png, label.png, product.glb, model_credit.json
  storyboard.json      the kept storyboard (and storyboard_16x9.json, storyboard_9x16.json)
  rounds/<i>/          each round: storyboard, frames, masks, sheet, filmstrip, cuts
  critiques/           <shot>.<round>.json, ad.<round>.json
  final/               <name>_16x9.mp4, <name>_9x16.mp4, music.wav
```

Model clients

Model	Used for
Muse Spark (Meta Model API)	Reading the brand from the photo; writing storyboards; revising them with tools
Muse Spark Open (Baseten)	The critic, by default: per-shot and whole-ad judgment
SAM 3.1 (Meta Model API)	Segmenting the product, cap and label
Muse Image (Meta Model API)	The test product photos of fictional brands

Every call goes through one layer: cached on disk by its request (a rerun costs nothing; a *sample* number asks the same again as a fresh answer), retried by the client, then outages waited out (10, 20, 40, 60 s), and recorded with its tokens and cost into the open run.

11. Results

Test	Result
An ad broken on purpose (light and framing)	1 of 5 shots passing → 5 of 5 in three revisions; 9 minutes, $0.48
A cut broken on purpose (view and light colour)	Whole ad 3 → 8 on continuity in one revision; $0.27
Fill as asked vs measured (5 templates × 3 elevations × 3 fills)	Within 0.06 of the request; nothing cut
Measured checks on 5 ads in 9:16	No cuts or title overlaps after the narrow-frame fixes
A full ad (analysis, storyboard, loop, two 1080p videos)	About 20–25 minutes on an Intel Iris Xe; about $0.20 on the Meta API plus Baseten calls

Muse Spark vs Muse Spark Open as the critic

19 shots from the five hand-made ads, the same frames and measurements, each judged twice by each model.

	Muse Spark	Muse Spark Open
Shots passed	21%	26%
Same verdict when judged twice	89%	84%
Average change in the lowest score between the two judgments	0.3	0.7
Time per critique	35 s	37 s
Answers that broke the schema	0	0

The two agree on pass or fail for 74% of shots, and every disagreement is near the pass mark. The critic runs on Muse Spark Open by default: as fast, nearly as consistent, and open weights.

12. Known limits

- **Hard cases don't always converge**: a white product on a bright set in a vertical frame didn't reach a full pass in three revisions; the best round is kept.
- **Critic noise near the pass mark**: the second opinion reduces it but doesn't remove it.
- **Model orientation** tells upright from lying down, not front from back.
- **Objaverse is noisy** (its categories were assigned automatically), and Sketchfab models often show real brands: a person picks, and demo ads use fictional brands.
- **Labels on downloaded models** aren't placed; a model keeps its own texture, tinted.
- **Nested transparency**: the renderer doesn't show a see-through surface through another one.
- **Reproducibility**: renders can differ by one level in a few pixels between Studio sessions, so a resumed run may judge a shot again.

13. Built during the hackathon

PBR_Engine existed before the event and is used as a prebuilt library; glass, depth of field, clear coat and specular were added to it during the hackathon. Everything in this repository was written during the hackathon, from October 27, 2026: the Studio library and program, every agent, the model clients, the catalogue, the final cut and music, the app, and the test photos (made with Muse Image).

Credits

- Fonts: Playfair Display, Montserrat, Oswald, Great Vibes (SIL Open Font License 1.1).
- HDRIs: Poly Haven, CC0.
- Downloaded 3D models: credited per run (CC0 or CC-BY).
- Every brand in the photos, ads and examples is fictional.